

DR-VERGE

View-Evidence Relational Grading Engine

Master Technical & Implementation Guide — Revisi v2

Dual-View Diabetic Retinopathy Grading via Complementarity-Shift Distillation

GEMASTIK XIX — Bidang VII, Karya Tulis Ilmiah TIK

Tim "Sunib et al."

Stanley Nathanael Wijaya · Seline Loewel · Ahmad Hamra

Dosen Pembimbing: Nikita Ananda Putri Misaling

Deadline Submission: 14 Agustus 2026

Dokumen ini adalah revisi v2 dari master guide DR-VERGE, disusun setelah review teknis eksternal menemukan sejumlah bug desain yang berpotensi membuat hasil eksperimen tidak valid meski kode berhasil berjalan. Setiap bagian yang direvisi ditandai kotak hijau (✓ diperbaiki). Setiap bagian yang secara jujur belum diverifikasi ditandai kotak kuning (⚠ butuh validasi) — dokumen ini TIDAK berpura-pura sudah sempurna; tujuannya adalah menjadi peta yang jujur tentang apa yang solid dan apa yang masih rawan.

Daftar Isi

0. Log Revisi v2 — Ringkasan Perubahan dari v1

Dokumen v1 direview secara teknis oleh anggota tim dan ditemukan **9 celah desain kritis** dan beberapa isu tambahan yang, jika tidak diperbaiki, dapat membuat hasil eksperimen tampak berhasil secara teknis (kode jalan tanpa error) namun **secara konseptual tidak valid** — terutama untuk klaim RQ1 (keunggulan CSD) dan RQ2 (dampak kuantisasi). Semua celah itu sudah dianalisis dan diperbaiki di dokumen ini. Ringkasannya:

#	Isu	Dampak jika tidak diperbaiki	Status v2
1	Baseline single-view (<code>view_mode</code>) sebenarnya tetap melatih <code>logit_dual</code> , bukan head single-view	Baseline macula-only/disc-only tidak valid; dual-view gain jadi bias	✓ Diperbaiki — Bagian 5, 8
2	Auxiliary head (macula/disc) tidak disupervisi pada kondisi <code>dual_no_distill</code> & <code>dual_logitkd</code>	Dual-view gain (G) pada kedua kondisi itu artifisial besar	✓ Diperbaiki — Bagian 6, 8
3	CSD memakai probabilitas cumulative threshold, bukan distribusi 5-kelas — klaim narasi perlu presisi	Klaim di paper bisa dianggap salah representasi jika tidak diluruskan	✓ Diperbaiki — Bagian 6.3, bahasa diluruskan
4	Softmax-KL pada CSD kehilangan informasi magnitude shift	CSD bisa 'menipu' dengan meniru arah tapi bukan besaran shift	✓ Diperbaiki — default diganti SmoothL1 / direction+magnitude, softmax-KL jadi varian ablasi
5	CORALHead tidak menjamin monotonicity antar-threshold	Δ (delta shift) bisa tercemar oleh ordinal violation, merusak sinyal CSD	✓ Diperbaiki — ordered-bias parameterization
6	Split disebut 'resmi' tapi kode men-generate split random sendiri	Klaim metodologi tidak konsisten, sulit dibandingkan ke paper lain	✓ Diperbaiki — split dinyatakan eksplisit sebagai custom, dengan validasi terbuka
7	Class imbalance (Grade 0 dominan) belum ditangani	Macro-F1 & sensitivity Grade 1-4 berisiko buruk meski QWK terlihat oke	✓ Diperbaiki — <code>pos_weight</code> per-threshold, konsisten di semua kondisi
8	Satu seed tidak cukup untuk klaim CSD lebih baik pada dataset sekecil DeepDRiD	Peningkatan QWK bisa cuma noise	✓ Diperbaiki — protokol 3-seed untuk 3 kondisi inti
9	Teacher tidak benar-benar menyimpan checkpoint terbaik (val QWK dihitung tapi tidak dipakai)	Teacher yang dipakai bisa epoch terakhir, bukan terbaik	✓ Diperbaiki — best-checkpoint + early stopping
10	Pretraining APTOS untuk backbone belum benar-benar terhubung ke training	Klaim strategi pretraining di paper tidak didukung kode nyata	✓ Diperbaiki — script <code>pretrain_aptos.py</code> nyata + loader eksplisit
11	Static PTQ eager-mode berisiko gagal tanpa <code>fuse_modules</code> & backend config	Fase kuantisasi bisa stuck berhari-hari untuk debugging	✓ Diperbaiki — <code>fuse_model()</code> , backend eksplisit, FX-mode sebagai fallback
12	<code>run_final_evaluation.py</code> rawan crash karena DictWriter dengan fieldnames tidak konsisten; teacher & INT8 belum masuk	Script evaluasi akhir bisa gagal total di hari terakhir	✓ Diperbaiki — schema CSV eksplisit + NaN-fill, semua 7 kondisi masuk

0.1 Isu Tambahan (Non-Kritis, Tetap Ditangani)

- Protokol pengukuran latency diperjelas: `batch=1`, `torch.set_num_threads` dilaporkan, median & p95 (bukan cuma mean), single-view dan dual-view diukur terpisah.
- Augmentasi macula & disc saat ini independen (rotasi/flip acak berbeda per foto) — ditandai sebagai keputusan desain yang butuh dikonfirmasi, bukan bug.
- Kapasitas backbone student ($\approx 0.3\text{--}0.4\text{M}$ parameter) ditandai sebagai risiko kapasitas — disiapkan opsi fallback (MobileNetV3-Small/Large, EfficientNet-B0).

0.2 Cara Membaca Dokumen Ini

Kotak **hijau (✓)** menandai bagian yang sudah direvisi mengikuti feedback dan dianggap solid secara desain — tapi tetap belum tentu bebas bug implementasi kecil (typo, dsb.) karena belum dieksekusi end-to-end pada data asli. Kotak **kuning (⚠)** menandai keputusan atau asumsi yang **secara eksplisit belum diverifikasi** dan wajib dicek di awal implementasi sebelum dipercaya. Anggap dokumen ini sebagai peta yang jujur soal confidence level tiap bagian, bukan jaminan seragam bahwa semuanya 100% benar.

1. Struktur Proyek & Environment

1.1 Struktur Folder (v2 — ditambah script yang sebelumnya hilang)

```

dr-verge/
├── configs/
│   ├── pretrain_aptos_resnet50.yaml      # NEW v2
│   ├── pretrain_aptos_lightweight.yaml  # NEW v2
│   ├── teacher.yaml
│   ├── student_macula_only.yaml
│   ├── student_disc_only.yaml
│   ├── student_dual_no_distill.yaml
│   ├── student_dual_logitkd.yaml
│   └── student_dual_csd.yaml
├── data/
│   ├── raw/
│   │   ├── deepdrid/
│   │   └── aptos2019/
│   └── splits/
│       ├── deepdrid_train.csv
│       ├── deepdrid_setB.csv
│       └── deepdrid_setC.csv
├── src/
│   ├── datasets.py
│   ├── models.py                      # direvisi: monotonic CORAL, forward_single, fuse_model
│   ├── losses.py                      # direvisi: pos_weight, aux loss, csd_variant
│   ├── pretrain_aptos.py              # NEW v2 — sebelumnya cuma komentar
│   ├── train_teacher.py               # direvisi: best-checkpoint, early stopping
│   ├── train_student.py               # direvisi: view_mode fix, multi-seed loop
│   ├── evaluate.py                   # direvisi: schema eksplisit, latency median/p95, OVR
│   ├── quantize.py                   # direvisi: fuse, backend, FX-mode fallback
│   ├── smoke_test.py                 # NEW v2 — wajib lolos sebelum full training
│   ├── aggregate_seeds.py             # NEW v2 — mean±std lintas seed
│   └── utils.py
├── checkpoints/
│   ├── pretrained_backbones/         # NEW v2
│   ├── teacher/
│   └── student/<condition_name>/seed_<n>/
├── results/
│   ├── metrics/
│   └── logs/
├── run_all_experiments.sh
└── requirements.txt

```

ini notes mending nanti ada 1 notebook full untuk full end-to-end pipeline deh biar bisa dijalanin selesai di colab / kaggle langsung sekaligus

Jadi tambahkan ada full_pipeline_notebook.ipynb di root aja

1.2 Dependencies & Setup

Tidak berubah dari v1 — lihat requirements.txt (torch, torchvision, scikit-learn, albumentations, pyyaml, tqdm, pandas). Tambahan satu poin: pastikan versi PyTorch mendukung torch.ao.quantization.quantize_fx (tersedia sejak torch ≥1.13, di-cover oleh torch>=2.2.0 yang sudah ditetapkan).

2. Spesifikasi Data & Pipeline Loading

2.1 Dataset & Split — Kejujuran soal "Resmi" vs "Custom"

⚠ **BUTUH VALIDASI LEBIH LANJUT** — Dokumen v1 menyebut split 300/100/100 sebagai 'split resmi' DeepDRiD, padahal kode `make_splits.py` melakukan random split sendiri dengan seed. Ini TIDAK BOLEH disebut split resmi kecuali dikonfirmasi dari metadata dataset bahwa partisi Set A/B/C memang sudah given per-pasien oleh penyedia dataset. Sebelum training dimulai, WAJIB dicek: apakah file metadata DeepDRiD punya kolom partisi eksplisit (mis. `original_split`)? Jika ya, pakai itu apa adanya. Jika tidak, split di dokumen ini adalah CUSTOM split, dan itu harus dinyatakan sejujurnya di paper — bukan diklaim sebagai split resmi challenge.

Untuk sisa dokumen ini, split diperlakukan sebagai **custom patient-wise split** dengan proporsi ~300/100/100 pasien, seed 42, kecuali ditemukan partisi resmi yang menggantikannya.

2.2 Dataset Utama: DeepDRiD & Pretraining: APTOS 2019

- DeepDRiD: 500 pasien, 2 mata/pasien, 2 foto/mata (macula, optic disc), grade ordinal 0–4 per mata. Wajib patient-wise split (dua mata pasien yang sama tidak boleh terpisah antar-set).
- APTOS 2019: 3.662 gambar single-view, dipakai HANYA untuk pretraining backbone (Bagian 7), bukan untuk evaluasi final.

⚠ **BUTUH VALIDASI LEBIH LANJUT** — Nama kolom CSV (`patient_id`, `macula_path`, `disc_path`, `grade`) adalah asumsi struktural, bukan hasil pengecekan langsung terhadap metadata asli DeepDRiD/APTOS. Sesuaikan `datasets.py` begitu file metadata asli diunduh dan diperiksa.

2.3 Preprocessing & Augmentasi

Sama seperti v1: crop border hitam, resize 224×224, normalisasi ImageNet. Augmentasi training: horizontal flip, rotasi $\pm 15^\circ$, brightness/contrast ringan.

⚠ **BUTUH VALIDASI LEBIH LANJUT** — Augmentasi saat ini dipanggil TERPISAH untuk macula dan disc (rotasi/flip acak berbeda per foto pada sampel yang sama). Ini bukan otomatis salah — kedua foto memang tidak pixel-aligned — tapi merupakan KEPUTUSAN yang harus dikonfirmasi tim, bukan default yang tidak disadari. Poin kedua yang lebih penting: **horizontal flip berpotensi mengubah makna klinis laterality (mata kiri vs kanan, posisi optic disc nasal/temporal)** — sebelum mengaktifkan flip, konfirmasi ke pembimbing/literatur apakah flip aman dipakai untuk foto fundus pada skema DR grading ini. Jika ragu, nonaktifkan `A.HorizontalFlip` sampai terkonfirmasi.

2.4 Penanganan Class Imbalance (BARU di v2)

✓ **DIPERBAIKI DI v2** — Ditambahkan penghitungan `pos_weight` per-threshold dari distribusi grade di training set, dipakai konsisten di SEMUA kondisi (teacher, semua varian student, semua seed) — supaya perbedaan performa antar-kondisi murni berasal dari metode distilasi, bukan dari sebagian kondisi kebetulan lebih tertolong oleh imbalance handling.

```
# src/utils.py – tambahan v2
import pandas as pd
import torch

def compute_pos_weights(train_csv: str, num_thresholds: int = 4) -> torch.Tensor:
    """
    Untuk tiap threshold k, pos_weight_k = N_negatif_k / N_positif_k,
```

```

dipakai sebagai argumen pos_weight pada
F.binary_cross_entropy_with_logits (lihat Bagian 6.1).
Dihitung SEKALI dari train split, dipakai sama untuk semua kondisi.
"""
df = pd.read_csv(train_csv)
grades = df["grade"].values
weights = []
for k in range(num_thresholds):
    pos = (grades > k).sum()
    neg = (grades <= k).sum()
    w = neg / max(pos, 1)
    weights.append(w)
return torch.tensor(weights, dtype=torch.float32)

```

⚠ **BUTUH VALIDASI LEBIH LANJUT** — Alternatif `WeightedRandomSampler` (resampling, bukan reweighting loss) belum diimplementasikan sebagai default. `Pos_weight` dipilih karena lebih sederhana dan cukup untuk BCE-based CORAL loss, tapi kalau distribusi grade di DeepDRiD ternyata SANGAT timpang (mis. Grade 0 >70%), pertimbangkan kombinasi keduanya — cek distribusi aktual dulu sebelum memutuskan.

2.5 Dataset Classes & Split Generator

Struktur class `DeepDRiDDualViewDataset` dan `APTOSSingleViewDataset` tidak berubah signifikan dari v1. `make_splits.py` diberi komentar eksplisit yang menegaskan sifatnya sebagai custom split (lihat 2.1):

```

# scripts/make_splits.py (v2 - komentar kejujuran ditambahkan)
import pandas as pd
from sklearn.model_selection import train_test_split

def make_patient_wise_split(metadata_csv, seed=42):
    """
    PENTING: Ini adalah CUSTOM split, BUKAN partisi resmi DeepDRiD --
    kecuali metadata_csv terbukti sudah memuat kolom partisi resmi
    (mis. 'official_split'). CEK DULU sebelum menjalankan fungsi ini.
    Jika partisi resmi tersedia, gunakan itu langsung, JANGAN panggil
    train_test_split di sini.
    """
    df = pd.read_csv(metadata_csv)

    if "official_split" in df.columns:
        df[df.official_split == "train"].to_csv("data/splits/deepdrid_train.csv",
index=False)
        df[df.official_split == "setB"].to_csv("data/splits/deepdrid_setB.csv", index=False)
        df[df.official_split == "setC"].to_csv("data/splits/deepdrid_setC.csv", index=False)
        print("Memakai partisi RESMI dari metadata.")
        return

    print("WARNING: tidak ditemukan kolom partisi resmi -- membuat CUSTOM split.")
    patients = df["patient_id"].unique()
    train_p, temp_p = train_test_split(patients, train_size=300, random_state=seed)
    setB_p, setC_p = train_test_split(temp_p, train_size=100, random_state=seed)

    df[df.patient_id.isin(train_p)].to_csv("data/splits/deepdrid_train.csv", index=False)
    df[df.patient_id.isin(setB_p)].to_csv("data/splits/deepdrid_setB.csv", index=False)
    df[df.patient_id.isin(setC_p)].to_csv("data/splits/deepdrid_setC.csv", index=False)

if __name__ == "__main__":
    make_patient_wise_split("data/raw/deepdrid/metadata.csv")

```

3. Arsitektur Model — CORAL Head, Teacher, dan Mode Single-View

3.1 CORALHead dengan Jaminan Monotonicity (BARU di v2)

✓ **DIPERBAIKI DI v2** — v1 memakai bias bebas per-threshold sehingga $P(y>1) > P(y>0)$ bisa terjadi (melanggar struktur ordinal). Karena seluruh novelty CSD dihitung dari selisih Delta antar-probabilitas threshold, pelanggaran ini bisa mencemari sinyal distilasi. v2 memakai **ordered-bias parameterization**: bias threshold ke-k diturunkan dari bias dasar dikurangi akumulasi langkah positif (via softplus), sehingga $P(y>0) \geq P(y>1) \geq P(y>2) \geq P(y>3)$ TERJAMIN SECARA STRUKTURAL oleh arsitektur, bukan diharapkan muncul dari training.

```
# src/models.py — CORALHead v2 (monotonic-by-construction)
import torch
import torch.nn as nn
import torch.nn.functional as F

class CORALHead(nn.Module):
    def __init__(self, in_dim: int, num_classes: int = 5):
        super().__init__()
        self.num_thresholds = num_classes - 1          # K-1 = 4
        self.fc = nn.Linear(in_dim, 1, bias=False)     # shared weight  $g(z) = w^T z$ 
        self.base_bias = nn.Parameter(torch.tensor(0.0))
        # K-2 free step-parameters -> softplus -> non-negative increments
        self.bias_steps = nn.Parameter(torch.zeros(self.num_thresholds - 1))

    def _ordered_biases(self):
        steps = F.softplus(self.bias_steps)            #  $\geq 0$ , panjang K-2
        cum = torch.cat([torch.zeros(1, device=steps.device), torch.cumsum(steps, dim=0)]) #
        panjang K-1
        biases = self.base_bias - cum                  # non-increasing di k
        return biases                                  # [K-1]

    def forward(self, z):
        g = self.fc(z)                                 # [B, 1]
        biases = self._ordered_biases().unsqueeze(0)   # [1, K-1]
        logits = g + biases                             # broadcast -> [B, K-1]
        probas = torch.sigmoid(logits)                  #  $P(y>k)$ , TERJAMIN
        non-increasing di k
        return logits, probas
```

Metrik sanity-check tambahan (Bagian 8) tetap disediakan — **Ordinal Violation Rate (OVR)** — untuk memverifikasi properti ini secara empiris dan untuk membandingkan terhadap versi non-monotonic pada ablation opsional jika waktu memungkinkan.

3.2 Teacher: Arsitektur & Mode Forward Tambahan

Struktur dasar teacher (ResNet-50 shared backbone + fusion + 3 CORAL head: main/macula/disc) tidak berubah dari v1. Yang ditambahkan di v2 adalah method `forward_single()`, dipakai untuk kebutuhan single-view yang jujur (Bagian 5) — baik untuk baseline single-view student, maupun untuk mengukur latency single-view teacher jika dibutuhkan sebagai referensi.

```
# src/models.py — Teacher v2
import torchvision.models as tv

class DualViewResNetTeacher(nn.Module):
    def __init__(self, num_classes: int = 5, feat_dim: int = 2048):
        super().__init__()
```

```

backbone = tv.resnet50(weights=tv.ResNet50_Weights.IMAGENET1K_V2)
backbone.fc = nn.Identity()
self.backbone = backbone # SHARED WEIGHTS antar view
self.fusion_bn = nn.BatchNorm1d(feats_dim * 2)
self.main_head = CORALHead(feats_dim * 2, num_classes)
self.macula_head = CORALHead(feats_dim, num_classes)
self.disc_head = CORALHead(feats_dim, num_classes)

def forward(self, macula, disc):
    z_m = self.backbone(macula)
    z_d = self.backbone(disc)
    z_fused = self.fusion_bn(torch.cat([z_m, z_d], dim=1))

    logit_dual, p_dual = self.main_head(z_fused)
    logit_m, p_m = self.macula_head(z_m)
    logit_d, p_d = self.disc_head(z_d)

    return {
        "p_dual": p_dual, "logit_dual": logit_dual,
        "p_macula": p_m, "logit_macula": logit_m,
        "p_disc": p_d, "logit_disc": logit_d,
    }

def forward_single(self, x, which: str = "macula"):
    """NEW v2 -- forward HANYA satu view, HANYA lewat backbone + head
    terkait (TIDAK lewat fusion_bn/main_head). Dipakai supaya baseline
    single-view benar-benar mengukur kapasitas satu-foto, termasuk
    untuk latency yang jujur (bukan model dual yang salah satu
    inputnya diabaikan)."""
    z = self.backbone(x)
    head = self.macula_head if which == "macula" else self.disc_head
    logit, p = head(z)
    return {"logit": logit, "p": p}

```

4. Arsitektur Model — Student

4.1 Backbone Ringan — Direstruktur agar Fuse-Friendly (v2)

✓ **DIPERBAIKI DI v2** — v1 memakai satu method `forward()` yang menggabungkan conv-bn-act secara prosedural, sulit di-fuse untuk static quantization (Bagian 10). v2 memecah tiap depthwise-separable block menjadi sub-layer bernama (dw, bn1, act1, pw, bn2, act2) dengan instance `'ReLU6'` terpisah di tiap titik (bukan satu instance dipakai ulang) — ini prasyarat teknis `torch.ao.quantization.fuse_modules`, karena fusion butuh identitas modul yang unik per titik fuse.

```
# src/models.py — Student backbone v2
class DepthwiseSeparableBlock(nn.Module):
    def __init__(self, in_ch, out_ch, stride=1):
        super().__init__()
        self.dw = nn.Conv2d(in_ch, in_ch, 3, stride=stride, padding=1, groups=in_ch,
bias=False)
        self.bn1 = nn.BatchNorm2d(in_ch)
        self.act1 = nn.ReLU6(inplace=True)          # instance TERPISAH -- wajib utk fuse
        self.pw = nn.Conv2d(in_ch, out_ch, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_ch)
        self.act2 = nn.ReLU6(inplace=True)          # instance TERPISAH

    def forward(self, x):
        x = self.act1(self.bn1(self.dw(x)))
        x = self.act2(self.bn2(self.pw(x)))
        return x

    def fuse(self):
        """Dipanggil sebelum static PTQ -- lihat Bagian 10."""
        torch.ao.quantization.fuse_modules(
            self, [
                ["dw", "bn1", "act1"], ["pw", "bn2", "act2"]
            ], inplace=True
        )

class LightweightBackbone(nn.Module):
    def __init__(self):
        super().__init__()
        self.stem_conv = nn.Conv2d(3, 16, 3, stride=2, padding=1, bias=False)
        self.stem_bn = nn.BatchNorm2d(16)
        self.stem_act = nn.ReLU6(inplace=True)

        self.blocks = nn.ModuleList([
            DepthwiseSeparableBlock(16, 24, stride=2),
            DepthwiseSeparableBlock(24, 24, stride=1),
            DepthwiseSeparableBlock(24, 40, stride=2),
            DepthwiseSeparableBlock(40, 40, stride=1),
            DepthwiseSeparableBlock(40, 56, stride=1),
        ])
        self.gap = nn.AdaptiveAvgPool2d(1)
        self.out_dim = 56

    def forward(self, x):
        x = self.stem_act(self.stem_bn(self.stem_conv(x)))
        for block in self.blocks:
            x = block(x)
        return self.gap(x).flatten(1)

    def fuse_model(self):
        """NEW v2 -- panggil sebelum PTQ (Bagian 10)."""
```

```

        torch.ao.quantization.fuse_modules(self, [["stem_conv", "stem_bn", "stem_act"]],
inplace=True)
        for block in self.blocks:
            block.fuse()

```

4.2 Student: forward() dan forward_single() (v2)

```

class DualViewLightStudent(nn.Module):
    def __init__(self, num_classes: int = 5):
        super().__init__()
        self.backbone = LightweightBackbone()
        feat_dim = self.backbone.out_dim
        self.fusion_bn = nn.BatchNorm1d(feat_dim * 2)
        self.main_head = CORALHead(feat_dim * 2, num_classes)
        self.macula_head = CORALHead(feat_dim, num_classes)
        self.disc_head = CORALHead(feat_dim, num_classes)

    def forward(self, macula, disc):
        z_m = self.backbone(macula)
        z_d = self.backbone(disc)
        z_fused = self.fusion_bn(torch.cat([z_m, z_d], dim=1))
        logit_dual, p_dual = self.main_head(z_fused)
        logit_m, p_m = self.macula_head(z_m)
        logit_d, p_d = self.disc_head(z_d)
        return {
            "p_dual": p_dual, "logit_dual": logit_dual,
            "p_macula": p_m, "logit_macula": logit_m,
            "p_disc": p_d, "logit_disc": logit_d,
        }

    def forward_single(self, x, which: str = "macula"):
        """NEW v2 -- identik prinsipnya dengan teacher.forward_single().
        Dipakai WAJIB untuk training & evaluasi baseline single-view
        (Bagian 5, 6) supaya benar-benar hanya satu foto yang diproses,
        termasuk untuk pengukuran latency yang jujur."""
        z = self.backbone(x)
        head = self.macula_head if which == "macula" else self.disc_head
        logit, p = head(z)
        return {"logit": logit, "p": p}

```

4.3 Risiko Kapasitas Student (ditandai eksplisit)

⚠ **BUTUH VALIDASI LEBIH LANJUT** — Backbone student hanya kira-kira 0,3-0,4 juta parameter — ini sangat kecil. Ada risiko nyata bahwa jika standard KD ATAU CSD sama-sama gagal meningkatkan performa dibanding no-distill, penyebabnya BUKAN metode distilasinya yang lemah, melainkan kapasitas backbone yang menjadi bottleneck. Mitigasi: siapkan backbone fallback yang established (MobileNetV3-Small, MobileNetV3-Large, atau EfficientNet-B0) sebagai Plan B jika di Hari 7-8 hasil dual_csd maupun dual_logitkd sama-sama tidak mengungguli dual_no_distill secara meyakinkan.

```

# Fallback backbone (opsional, Plan B) -- src/models.py
import torchvision.models as tv

class MobileNetV3SmallBackbone(nn.Module):
    def __init__(self):
        super().__init__()
        m = tv.mobilenet_v3_small(weights=tv.MobileNet_V3_Small_Weights.IMAGENET1K_V1)
        self.features = m.features
        self.gap = nn.AdaptiveAvgPool2d(1)
        self.out_dim = 576 # sesuaikan dgn output channel terakhir MobileNetV3-Small

```

```
def forward(self, x):  
    x = self.features(x)  
    return self.gap(x).flatten(1)  
# forward_single & fuse_model mengikuti pola yang sama seperti LightweightBackbone
```

5. Spesifikasi Matematis Lengkap (Revisi v2)

Bagian ini adalah rujukan formula tunggal, sudah direvisi mengikuti seluruh feedback teknis. **Ini bagian paling penting di dokumen** — kode HARUS mengikuti notasi di sini.

5.1 CORAL Ordinal Loss dengan Class-Imbalance Weighting

Untuk $K = 5$ kelas (Grade 0..4), $K-1 = 4$ threshold biner.
Label biner turunan: $y_k = 1[y > k]$, $k = 0..3$

CORALHead (Bagian 3.1) menjamin $P(y>0) \geq P(y>1) \geq P(y>2) \geq P(y>3)$ secara struktural (ordered-bias parameterization).

Loss, dengan pos_weight_k per-threshold (Bagian 2.4) untuk menangani class imbalance:

$$L_{\text{ordinal}} = -(1/4) * \sum_{k=0}^3 [\text{pos_weight_k} * y_k * \log(P(y>k|z)) + (1 - y_k) * \log(1 - P(y>k|z))]$$

pos_weight_k DIHITUNG SEKALI dari train split (Bagian 2.4) dan DIPAKAI SAMA di teacher, semua varian student, dan semua seed -- supaya perbedaan hasil antar-kondisi murni dari metode distilasi, bukan dari sebagian kondisi kebetulan lebih terbantu imbalance handling.

Prediksi grade akhir: $y_{\text{hat}} = \sum_k 1[P(y>k|z) > 0.5]$

5.2 Auxiliary Supervision untuk Semua Kondisi Dual-View (BARU di v2)

✓ **DIPERBAIKI DI v2** — Di v1, hanya kondisi `dual_csd` yang menyupervisi `macula_head/disc_head` (secara tidak langsung lewat CSD). Kondisi `dual_no_distill` dan `dual_logitkd` membiarkan kedua auxiliary head TIDAK PERNAH dilatih, padahal `compute_dual_view_gain()` tetap membaca output keduanya -- membuat dual-view gain (G) pada kedua kondisi itu bias secara artifisial (Critical Issue 2). v2 mewajibkan auxiliary loss pada SEMUA kondisi dual-view, termasuk teacher, dengan bobot yang sama, sehingga perbandingan G antar-kondisi jadi adil.

Untuk SEMUA model dual-view (teacher, `dual_no_distill`, `dual_logitkd`, `dual_csd`) -- BUKAN untuk baseline single-view:

$$L_{\text{aux}} = L_{\text{ordinal}}(y, \text{logit_macula}) + L_{\text{ordinal}}(y, \text{logit_disc})$$

$$L_{\text{task_dual}} = L_{\text{ordinal}}(y, \text{logit_dual}) + \text{lambda_aux} * L_{\text{aux}}$$

lambda_aux (default 0.5) SAMA untuk seluruh kondisi dual-view.

5.3 Standard Logit Knowledge Distillation ($L_{\text{logit_KD}}$)

Tidak berubah secara matematis dari v1 — student meniru distribusi probabilitas dual-view teacher dengan temperature scaling:

$$\begin{aligned} p_{\text{soft}}^T &= \text{sigma}(\text{logit_dual}^T / \tau), & p_{\text{soft}}^S &= \text{sigma}(\text{logit_dual}^S / \tau) \\ L_{\text{logit_KD}} &= \text{BCE}(p_{\text{soft}}^T.\text{detach}(), p_{\text{soft}}^S) \\ \tau \text{ default} &= 2.0 \end{aligned}$$

5.4 Complementarity-Shift Distillation (L_CSD) — Revisi v2

⚠ **BUTUH VALIDASI LEBIH LANJUT** — Bahasa 'satu dimensi per grade' pada dokumen ide awal (chat) TIDAK berlaku untuk implementasi ini. Delta di sini adalah **pergeseran pada cumulative ordinal decision boundaries** (dimensi [B,4], satu per threshold), BUKAN pergeseran pada distribusi probabilitas 5 kelas. Klaim di paper WAJIB memakai istilah 'cumulative threshold shift' atau 'ordinal decision boundary shift', bukan 'per-grade probability shift', supaya presisi dan tidak overclaim. Opsi konversi ke distribusi 5-kelas eksplisit (q0..q4 dari selisih threshold berurutan) tersedia sebagai perluasan masa depan, TAPI membutuhkan penanganan tambahan karena monotonicity yang dijamin arsitektur (Bagian 3.1) membuat konversi ini valid secara matematis ($q_i \geq 0$ terjamin) -- namun ini BELUM diimplementasikan/divalidasi di v2, jadi tetap ditandai sebagai belum tervalidasi kalau mau dipakai.

5.4.1 Definisi Delta (tidak berubah secara struktural dari v1)

```
p_agg^T = ( p_macula^T + p_disc^T ) / 2          [B,4]
delta^T = p_dual^T - p_agg^T                    [B,4]

p_agg^S = ( p_macula^S + p_disc^S ) / 2          [B,4]
delta^S = p_dual^S - p_agg^S                    [B,4]

Semua komponen sisi teacher (p_dual^T, p_macula^T, p_disc^T) di-detach()
-- gradien HANYA mengalir ke sisi student.
```

5.4.2 Tiga Varian Loss CSD (BARU di v2 — default berubah)

✓ **DIPERBAIKI DI v2** — v1 hanya memakai softmax-KL, yang terbukti kehilangan informasi magnitude (Critical Issue 4): softmax menormalkan skala, sehingga dua delta dengan magnitude sangat berbeda (mis. [-0.01,0,0.01,0] vs [-0.1,0,0.1,0]) bisa menghasilkan distribusi softmax yang mirip. v2 menjadikan **SmoothL1 (signed, tanpa normalisasi)** sebagai DEFAULT, dan menyediakan varian direction+magnitude serta softmax-KL (v1) sebagai opsi ablasi eksplisit -- dipilih lewat `csd_variant` di config.

```
VARIAN 1 (DEFAULT v2) -- signed SmoothL1, mempertahankan magnitude:
    L_CSD = SmoothL1( delta^S, delta^T )

VARIAN 2 -- direction + magnitude terpisah:
    L_direction = 1 - cos_similarity( delta^S, delta^T )
    L_magnitude = SmoothL1( delta^S, delta^T )
    L_CSD = 0.5 * L_direction + 0.5 * L_magnitude

VARIAN 3 (v1, dipertahankan sbg ABLASI, bukan default) -- softmax-KL:
    L_CSD = KL( softmax(delta^T / tau_csd) || softmax(delta^S / tau_csd) )
```

Ketiganya WAJIB dibandingkan lewat ablation di Set B (Bagian 8) sebelum memutuskan varian final untuk dilaporkan sebagai DR-VERGE utama -- klaim "CSD lebih baik" HARUS didasarkan pada varian yang benar-benar menang secara empiris, bukan pada varian default semata.

5.5 Loss Gabungan Student — Revisi Lengkap (v2)

✓ **DIPERBAIKI DI v2** — Perbaikan paling kritis (Critical Issue 1): v1 SELALU memakai `student_out['logit_dual']` untuk `L_ordinal`, TIDAK PEDULI `view_mode`-nya apa -- akibatnya baseline `macula_only/disc_only` sebenarnya tetap melatih head dual-view, bukan head single-view yang seharusnya dievaluasi. v2 memilih task logit berdasarkan `view_mode` secara eksplisit, dan untuk mode single-view memakai `forward_single()` (Bagian 3-4) sehingga backbone HANYA memproses satu foto -- bukan model dual yang salah satu inputnya diabaikan begitu saja.

FUNGSI PEMILIHAN OUTPUT BERDASARKAN `view_mode`:

```

view_mode == "dual":
    student_out = student.forward(macula, disc)
    task_logit = student_out["logit_dual"]
    -- L_aux (Bagian 5.2) DITERAPKAN
    -- alpha/beta (KD/CSD) DITERAPKAN jika >0

view_mode == "macula_only":
    student_out = student.forward_single(macula, which="macula")
    task_logit = student_out["logit"]
    -- TIDAK ADA L_aux, TIDAK ADA KD/CSD (single-view tidak
    punya pasangan view untuk didistilasi dgn cara yg sama)

view_mode == "disc_only":
    student_out = student.forward_single(disc, which="disc")
    task_logit = student_out["logit"]

LOSS AKHIR:
L_student = L_ordinal(y, task_logit, pos_weight)
    + [ hanya jika view_mode == "dual" ]:
        lambda_aux * L_aux
        + alpha * L_logit_KD      (jika alpha > 0)
        + beta * L_CSD            (jika beta > 0)

Default: lambda_aux=0.5, alpha=0.5, beta=0.5 (CSD default variant
SmoothL1, Bagian 5.4.2), tau_kd=2.0.
# src/losses.py – v2 lengkap
import torch
import torch.nn as nn
import torch.nn.functional as F

def coral_loss(logits, labels, num_thresholds=4, pos_weight=None):
    device = logits.device
    levels = torch.arange(num_thresholds, device=device).unsqueeze(0)
    y_k = (labels.unsqueeze(1) > levels).float()
    return F.binary_cross_entropy_with_logits(logits, y_k, pos_weight=pos_weight)

def aux_loss(student_out, labels, pos_weight=None):
    l_m = coral_loss(student_out["logit_macula"], labels, pos_weight=pos_weight)
    l_d = coral_loss(student_out["logit_disc"], labels, pos_weight=pos_weight)
    return l_m + l_d

def logit_kd_loss(logit_dual_teacher, logit_dual_student, tau=2.0):
    p_t = torch.sigmoid(logit_dual_teacher.detach() / tau)
    p_s = torch.sigmoid(logit_dual_student / tau)
    return F.binary_cross_entropy(p_s, p_t)

def csd_loss(p_dual_t, p_macula_t, p_disc_t, p_dual_s, p_macula_s, p_disc_s,
             variant="smoothl1", tau_csd=0.5):
    p_agg_t = (p_macula_t.detach() + p_disc_t.detach()) / 2
    delta_t = p_dual_t.detach() - p_agg_t
    p_agg_s = (p_macula_s + p_disc_s) / 2
    delta_s = p_dual_s - p_agg_s

    if variant == "smoothl1":
        return F.smooth_l1_loss(delta_s, delta_t)
    elif variant == "direction_magnitude":
        cos_sim = F.cosine_similarity(delta_s, delta_t, dim=1, eps=1e-6)
        l_dir = (1 - cos_sim).mean()

```

```

        l_mag = F.smooth_l1_loss(delta_s, delta_t)
        return 0.5 * l_dir + 0.5 * l_mag
    elif variant == "kl_softmax":
        # v1, ablasi saja
        log_q = F.log_softmax(delta_s / tau_csd, dim=1)
        p_target = F.softmax(delta_t / tau_csd, dim=1)
        return F.kl_div(log_q, p_target, reduction="batchmean")
    else:
        raise ValueError(f"unknown csd_variant: {variant}")

def get_student_output(student, macula, disc, view_mode):
    """NEW v2 -- satu-satunya tempat pemilihan forward berdasarkan
    view_mode, dipakai training maupun evaluasi supaya konsisten."""
    if view_mode == "dual":
        return student(macula, disc)
    elif view_mode == "macula_only":
        return student.forward_single(macula, which="macula")
    elif view_mode == "disc_only":
        return student.forward_single(disc, which="disc")
    raise ValueError(f"unknown view_mode: {view_mode}")

def combined_student_loss(teacher_out, student_out, labels, view_mode,
                          alpha=0.0, beta=0.0, lambda_aux=0.5,
                          tau_kd=2.0, csd_variant="smoothl1", tau_csd=0.5,
                          pos_weight=None):
    if view_mode == "dual":
        task_logit = student_out["logit_dual"]
    else:
        task_logit = student_out["logit"]

    l_task = coral_loss(task_logit, labels, pos_weight=pos_weight)
    total = l_task
    log = {"L_task": l_task.item()}

    if view_mode == "dual":
        l_aux = aux_loss(student_out, labels, pos_weight=pos_weight)
        total = total + lambda_aux * l_aux
        log["L_aux"] = l_aux.item()

    if alpha > 0:
        l_kd = logit_kd_loss(teacher_out["logit_dual"], student_out["logit_dual"],
tau_kd)
        total = total + alpha * l_kd
        log["L_logit_KD"] = l_kd.item()

    if beta > 0:
        l_csd = csd_loss(
            teacher_out["p_dual"], teacher_out["p_macula"], teacher_out["p_disc"],
            student_out["p_dual"], student_out["p_macula"], student_out["p_disc"],
            variant=csd_variant, tau_csd=tau_csd,
        )
        total = total + beta * l_csd
        log["L_CSD"] = l_csd.item()

    log["L_total"] = total.item()
    return total, log

```

5.6 Ordinal Violation Rate (OVR) — Metrik Sanity-Check (BARU v2)

Dengan CORALHead v2 (ordered-bias), OVR terjamin = 0 secara matematis. Dipakai untuk (a) sanity-check implementasi -- kalau OVR > 0, ada bug, dan (b) sebagai titik pembandingan kalau tim menjalankan ablasi opsional dengan CORALHead non-monotonic versi lama.

```
OVR = mean( 1[ P(y>k+1) > P(y>k) ] ), untuk semua k dan semua sampel
def ordinal_violation_rate(p):
    """p: [B, K-1] probabilitas cumulative, idealnya non-increasing."""
    diffs = p[:, 1:] - p[:, :-1]
    violations = (diffs > 0).float()
    return violations.mean().item()
```

5.7 Dual-View Gain (metrik evaluasi)

⚠ **BUTUH VALIDASI LEBIH LANJUT** — $G = QWK_dual - \max(QWK_macula, QWK_disc)$ HANYA valid dibandingkan antar-kondisi jika seluruh kondisi dual-view sama-sama menyupervisi auxiliary head (Bagian 5.2 sudah menjamin ini di v2). Sebelum melaporkan G di paper, verifikasi dulu lewat Gate 2 (Bagian 14) bahwa QWK_macula dan QWK_disc teacher masuk akal (bukan mendekati performa acak) -- kalau auxiliary head teacher sendiri lemah, seluruh sinyal G dan CSD yang diturunkan darinya jadi tidak bermakna.

6. Pipeline Training — Tahap 1 (Teacher), Revisi v2

6.1 Perbaikan Best-Checkpoint & Early Stopping (BARU v2)

✓ **DIPERBAIKI DI v2** — v1 menghitung `best_val_loss` tapi tidak pernah memakainya untuk menyimpan checkpoint -- model yang tersimpan adalah hasil epoch TERAKHIR, bukan TERBAIK (Critical Issue 9). v2 menyimpan checkpoint berdasarkan **val QWK tertinggi** (bukan val loss, karena QWK adalah metrik utama yang benar-benar dipedulikan), dengan early stopping patience.

```
# src/train_teacher.py - v2
import torch
from torch.utils.data import DataLoader
from tqdm import tqdm
from sklearn.metrics import cohen_kappa_score

from datasets import DeepDRiDDualViewDataset, train_transform, eval_transform
from models import DualViewResNetTeacher
from losses import coral_loss, aux_loss
from utils import set_seed, load_config, get_device, compute_pos_weights

def train_teacher(config_path="configs/teacher.yaml"):
    cfg = load_config(config_path)
    set_seed(cfg["seed"])
    device = get_device()

    pos_weight = compute_pos_weights(cfg["train_csv"]).to(device)  # NEW v2, Bagian 2.4

    train_ds = DeepDRiDDualViewDataset(cfg["train_csv"], cfg["root_dir"], train_transform)
    val_ds = DeepDRiDDualViewDataset(cfg["val_csv"], cfg["root_dir"], eval_transform)
    train_loader = DataLoader(train_ds, batch_size=cfg["batch_size"], shuffle=True,
num_workers=4)
    val_loader = DataLoader(val_ds, batch_size=cfg["batch_size"], shuffle=False,
num_workers=4)

    model = DualViewResNetTeacher(num_classes=5).to(device)

    # NEW v2 -- pretraining APTOS benar-benar dimuat (Bagian 7), bukan komentar
    if cfg.get("aptos_backbone_ckpt"):
        model.backbone.load_state_dict(torch.load(cfg["aptos_backbone_ckpt"],
map_location=device))
        print(f"Backbone teacher dimuat dari {cfg['aptos_backbone_ckpt']}")

    lambda_aux = cfg.get("lambda_aux", 0.5)

    for p in model.backbone.parameters():
        p.requires_grad = False
    opt = torch.optim.Adam([p for p in model.parameters() if p.requires_grad], lr=1e-3)
    best_qwk = _run_epochs(model, train_loader, val_loader, opt, device,
                           epochs=cfg["freeze_epochs"], pos_weight=pos_weight,
                           lambda_aux=lambda_aux, ckpt_path=cfg["output_ckpt"],
                           patience=cfg.get("patience", 5), best_qwk=-1.0)

    model.load_state_dict(torch.load(cfg["output_ckpt"], map_location=device)["model_state"])
    for p in model.backbone.parameters():
        p.requires_grad = True
    opt = torch.optim.Adam(model.parameters(), lr=1e-5)
    best_qwk = _run_epochs(model, train_loader, val_loader, opt, device,
                           epochs=cfg["finetune_epochs"], pos_weight=pos_weight,
                           lambda_aux=lambda_aux, ckpt_path=cfg["output_ckpt"],
```

```

        patience=cfg.get("patience", 5), best_qwk=best_qwk)

    print(f"Teacher training selesai. Best val QWK = {best_qwk:.4f}")

def _run_epochs(model, train_loader, val_loader, opt, device, epochs,
                pos_weight, lambda_aux, ckpt_path, patience, best_qwk):
    patience_counter = 0
    for epoch in range(epochs):
        model.train()
        for batch in tqdm(train_loader, desc=f"epoch {epoch}"):
            macula, disc, y = batch["macula"].to(device), batch["disc"].to(device),
batch["label"].to(device)
            out = model(macula, disc)
            loss = (coral_loss(out["logit_dual"], y, pos_weight=pos_weight)
                    + lambda_aux * aux_loss(out, y, pos_weight=pos_weight))    # NEW v2: aux
loss
            opt.zero_grad(); loss.backward(); opt.step()

            val_qwk = _validate_qwk(model, val_loader, device)
            print(f"epoch {epoch}: val_QWK={val_qwk:.4f}")

            if val_qwk > best_qwk:                                # NEW v2: benar2 disimpan
                best_qwk = val_qwk
                patience_counter = 0
                torch.save({"model_state": model.state_dict(), "epoch": epoch, "val_qwk":
val_qwk}, ckpt_path)
            else:
                patience_counter += 1
                if patience_counter >= patience:
                    print(f"Early stopping di epoch {epoch} (patience={patience})")
                    break
    return best_qwk

@torch.no_grad()
def _validate_qwk(model, loader, device):
    model.eval()
    preds, targets = [], []
    for batch in loader:
        macula, disc, y = batch["macula"].to(device), batch["disc"].to(device),
batch["label"]
        out = model(macula, disc)
        grade_pred = (out["p_dual"] > 0.5).sum(dim=1)
        preds.extend(grade_pred.cpu().tolist())
        targets.extend(y.tolist())
    return cohen_kappa_score(targets, preds, weights="quadratic")

if __name__ == "__main__":
    train_teacher()

```

6.2 Config Teacher (v2 — menambah pos_weight & patience implisit lewat compute_pos_weights)

```

seed: 42
train_csv: data/splits/deepdrid_train.csv
val_csv: data/splits/deepdrid_setB.csv
root_dir: data/raw/deepdrid
batch_size: 16
freeze_epochs: 5

```

```
finetune_epochs: 15  
patience: 5  
lambda_aux: 0.5  
aptos_backbone_ckpt: checkpoints/pretrained_backbones/aptos_resnet50_backbone.pt  
output_ckpt: checkpoints/teacher/teacher_final.pt
```

7. Pipeline Pretraining APTOS (BARU v2)

✓ **DIPERBAIKI DI v2** — v1 hanya menulis `# model.backbone.load_state_dict(...)` sebagai komentar -- pipeline pretraining yang dijanjikan di narasi metodologi TIDAK benar-benar ada di kode (Critical Issue 10). v2 menyediakan script nyata, dan secara eksplisit memisahkan pretraining untuk teacher (ResNet-50) dan student (LightweightBackbone) KARENA KEDUANYA ARSITEKTUR BERBEDA -- checkpoint backbone TIDAK BISA dipakai bergantian antara keduanya.

```
# src/pretrain_aptos.py — NEW v2
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from tqdm import tqdm
from sklearn.metrics import cohen_kappa_score

from datasets import APTOSSingleViewDataset, train_transform, eval_transform
from models import CORALHead, LightweightBackbone
import torchvision.models as tv
from utils import set_seed, load_config, get_device, compute_pos_weights

def build_backbone(backbone_type: str):
    if backbone_type == "resnet50":
        m = tv.resnet50(weights=tv.ResNet50_Weights.IMAGENET1K_V2)
        m.fc = nn.Identity()
        return m, 2048
    elif backbone_type == "lightweight":
        m = LightweightBackbone()
        return m, m.out_dim
    raise ValueError(backbone_type)

def pretrain_backbone(config_path):
    cfg = load_config(config_path)
    set_seed(cfg["seed"])
    device = get_device()
    pos_weight = compute_pos_weights(cfg["train_csv"]).to(device)

    train_ds = APTOSSingleViewDataset(cfg["train_csv"], cfg["root_dir"], train_transform)
    val_ds = APTOSSingleViewDataset(cfg["val_csv"], cfg["root_dir"], eval_transform)
    train_loader = DataLoader(train_ds, batch_size=cfg["batch_size"], shuffle=True,
num_workers=4)
    val_loader = DataLoader(val_ds, batch_size=cfg["batch_size"], shuffle=False,
num_workers=4)

    backbone, feat_dim = build_backbone(cfg["backbone_type"])
    head = CORALHead(feat_dim, num_classes=5)
    model = nn.ModuleDict({"backbone": backbone, "head": head}).to(device)

    opt = torch.optim.Adam(model.parameters(), lr=cfg["lr"])
    best_qwk = -1.0

    for epoch in range(cfg["epochs"]):
        model.train()
        for batch in tqdm(train_loader, desc=f"[pretrain-{cfg['backbone_type']}]" epoch
{epoch}"):
            img, y = batch["image"].to(device), batch["label"].to(device)
            z = model["backbone"](img)
            logit, _ = model["head"](z)
            from losses import coral_loss
            loss = coral_loss(logit, y, pos_weight=pos_weight)
```

```

        opt.zero_grad(); loss.backward(); opt.step()

        val_qwk = _validate(model, val_loader, device)
        print(f"epoch {epoch}: val_QWK={val_qwk:.4f}")
        if val_qwk > best_qwk:
            best_qwk = val_qwk
            torch.save(model["backbone"].state_dict(), cfg["output_backbone_ckpt"]) #
backbone-ONLY

        print(f"Pretraining {cfg['backbone_type']} selesai. Best val QWK = {best_qwk:.4f}")

@torch.no_grad()
def _validate(model, loader, device):
    model.eval()
    preds, targets = [], []
    for batch in loader:
        img, y = batch["image"].to(device), batch["label"]
        z = model["backbone"](img)
        logit, p = model["head"](z)
        grade_pred = (p > 0.5).sum(dim=1)
        preds.extend(grade_pred.cpu().tolist())
        targets.extend(y.tolist())
    return cohen_kappa_score(targets, preds, weights="quadratic")

if __name__ == "__main__":
    import sys
    pretrain_backbone(sys.argv[1]) # configs/pretrain_aptos_resnet50.yaml ATAU
_lightweight.yaml

```

7.1 Dua Config Terpisah

```

# configs/pretrain_aptos_resnet50.yaml -- untuk TEACHER
backbone_type: resnet50
seed: 42
train_csv: data/splits/aptos_train.csv
val_csv: data/splits/aptos_val.csv
root_dir: data/raw/aptos2019
batch_size: 32
lr: 0.0001
epochs: 20
output_backbone_ckpt: checkpoints/pretrained_backbones/aptos_resnet50_backbone.pt

# ---
# configs/pretrain_aptos_lightweight.yaml -- untuk STUDENT
backbone_type: lightweight
seed: 42
train_csv: data/splits/aptos_train.csv
val_csv: data/splits/aptos_val.csv
root_dir: data/raw/aptos2019
batch_size: 32
lr: 0.001
epochs: 30
output_backbone_ckpt: checkpoints/pretrained_backbones/aptos_lightweight_backbone.pt

```

Kedua script ini dijalankan di **Hari 2** (Bagian 15), SEBELUM training teacher/student dimulai — output-nya (aptos_resnet50_backbone.pt dan aptos_lightweight_backbone.pt) menjadi prasyarat wajib untuk Bagian 6 dan Bagian 8.

8. Pipeline Training — Tahap 2 (Student), Revisi Kritis v2

8.1 Perbaikan Utama: view_mode Sekarang Benar-Benar Dipakai

✓ **DIPERBAIKI DI v2** — Ini adalah perbaikan paling penting di seluruh dokumen (Critical Issue 1). v1 SELALU melatih `student_out['logit_dual']` di dalam `combined_student_loss`, apa pun `view_mode`-nya -- akibatnya kondisi `macula_only` dan `disc_only` sebenarnya melatih model DUAL-VIEW, lalu saat evaluasi baru membaca `p_macula/p_disc` yang hampir random karena tidak pernah disupervisi langsung. v2 memakai `get_student_output()` (Bagian 5.5) yang memanggil `forward_single()` untuk mode single-view, sehingga backbone benar-benar HANYA memproses satu foto, dan loss benar-benar melatih head yang relevan.

8.2 Protokol Multi-Seed (BARU v2)

✓ **DIPERBAIKI DI v2** — v1 memakai seed tunggal (42) untuk semua kondisi. Mengingat DeepDRiD kecil (~2.000 gambar) dan variance antar-run bisa besar, satu seed TIDAK CUKUP untuk mengklaim CSD lebih baik dari baseline (Critical Issue 8) -- peningkatan QWK 0.01-0.02 pada satu seed bisa jadi murni noise. v2 mewajibkan **3 seed (42, 123, 2026)** untuk tiga kondisi inti yang jadi jawaban RQ1: `dual_no_distill`, `dual_logitkd`, `dual_csd`. Hasil dilaporkan sebagai mean $\pm$ std, bukan angka tunggal.

- `teacher`, `macula_only`, `disc_only`: 1 seed cukup (bukan bagian klaim komparatif utama RQ1).
- `dual_csd_int8_ptq`: 1 seed cukup (PTQ deterministik terhadap checkpoint yang sudah tetap, bukan sumber variance training).
- `dual_no_distill`, `dual_logitkd`, `dual_csd`: **WAJIB 3 seed**, dilaporkan mean $\pm$ std di tabel hasil paper.

8.3 Script Training Student (v2 Lengkap)

```
# src/train_student.py - v2
import torch
from torch.utils.data import DataLoader
from tqdm import tqdm
from sklearn.metrics import cohen_kappa_score

from datasets import DeepDRiDDualViewDataset, train_transform, eval_transform
from models import DualViewResNetTeacher, DualViewLightStudent
from losses import combined_student_loss, get_student_output
from utils import set_seed, load_config, get_device, compute_pos_weights

def train_student(config_path, seed_override=None):
    cfg = load_config(config_path)
    seed = seed_override if seed_override is not None else cfg["seed"]
    set_seed(seed)
    device = get_device()
    pos_weight = compute_pos_weights(cfg["train_csv"]).to(device)

    train_ds = DeepDRiDDualViewDataset(cfg["train_csv"], cfg["root_dir"], train_transform)
    val_ds = DeepDRiDDualViewDataset(cfg["val_csv"], cfg["root_dir"], eval_transform)
    train_loader = DataLoader(train_ds, batch_size=cfg["batch_size"], shuffle=True,
num_workers=4)
    val_loader = DataLoader(val_ds, batch_size=cfg["batch_size"], shuffle=False,
num_workers=4)

    teacher = DualViewResNetTeacher(num_classes=5).to(device)
    teacher.load_state_dict(torch.load(cfg["teacher_ckpt"],
map_location=device)["model_state"])
```

```

teacher.eval()
for p in teacher.parameters():
    p.requires_grad = False

student = DualViewLightStudent(num_classes=5).to(device)
if cfg.get("student_backbone_ckpt"):
    # NEW v2 -- APTOS pretrain
nyata
    student.backbone.load_state_dict(torch.load(cfg["student_backbone_ckpt"],
map_location=device))
    print(f"Backbone student dimuat dari {cfg['student_backbone_ckpt']}")

opt = torch.optim.Adam(student.parameters(), lr=cfg["lr"])

view_mode = cfg["view_mode"]
alpha, beta = cfg.get("alpha", 0.0), cfg.get("beta", 0.0)
lambda_aux = cfg.get("lambda_aux", 0.5)
csd_variant = cfg.get("csd_variant", "smoothl1")
KL
    # NEW v2, default sudah bukan

ckpt_path = cfg["output_ckpt"].replace(".pt", f"_seed{seed}.pt")
per-seed
    # NEW v2 -- checkpoint

best_qwk, patience_counter = -1.0, 0

for epoch in range(cfg["epochs"]):
    student.train()
    for batch in tqdm(train_loader, desc=f"[{cfg['run_name']}]|seed{seed}] epoch
{epoch}"):
        macula, disc, y = batch["macula"].to(device), batch["disc"].to(device),
batch["label"].to(device)

        with torch.no_grad():
            teacher_out = teacher(macula, disc)
            # tetap dual, teacher selalu
penuh

            student_out = get_student_output(student, macula, disc, view_mode)
            # NEW v2 --
FIX UTAMA

            loss, log = combined_student_loss(
                teacher_out, student_out, y, view_mode,
                alpha=alpha, beta=beta, lambda_aux=lambda_aux,
                tau_kd=cfg.get("tau_kd", 2.0), csd_variant=csd_variant,
                tau_csd=cfg.get("tau_csd", 0.5), pos_weight=pos_weight,
            )
            opt.zero_grad(); loss.backward(); opt.step()

val_qwk = _validate_qwk(student, val_loader, device, view_mode)
print(f"epoch {epoch}: val_QWK={val_qwk:.4f}")

if val_qwk > best_qwk:
    best_qwk = val_qwk
    patience_counter = 0
    torch.save({"model_state": student.state_dict(), "epoch": epoch,
                "val_qwk": val_qwk, "seed": seed}, ckpt_path)
else:
    patience_counter += 1
    if patience_counter >= cfg.get("patience", 5):
        print(f"Early stopping epoch {epoch}"); break

print(f"[{cfg['run_name']}]|seed{seed}] best val QWK = {best_qwk:.4f} -> {ckpt_path}")
return best_qwk, ckpt_path

```

```

@torch.no_grad()
def _validate_qwk(student, loader, device, view_mode):
    student.eval()
    preds, targets = [], []
    for batch in loader:
        macula, disc, y = batch["macula"].to(device), batch["disc"].to(device),
batch["label"]
        out = get_student_output(student, macula, disc, view_mode)
        p_key = "p_dual" if view_mode == "dual" else "p"
        grade_pred = (out[p_key] > 0.5).sum(dim=1)
        preds.extend(grade_pred.cpu().tolist())
        targets.extend(y.tolist())
    return cohen_kappa_score(targets, preds, weights="quadratic")

if __name__ == "__main__":
    import sys
    config_path = sys.argv[1]
    seeds = [int(s) for s in sys.argv[2].split(",")] if len(sys.argv) > 2 else [None]
    for s in seeds:
        train_student(config_path, seed_override=s)

```

8.4 Config per Kondisi (v2 — menambah csd_variant & student_backbone_ckpt)

```

# configs/student_dual_csd.yaml <-- DR-VERGE PENUH
run_name: dual_csd
seed: 42
train_csv: data/splits/deepdrid_train.csv
val_csv: data/splits/deepdrid_setB.csv
root_dir: data/raw/deepdrid
batch_size: 16
lr: 0.001
epochs: 30
patience: 5
teacher_ckpt: checkpoints/teacher/teacher_final.pt
student_backbone_ckpt: checkpoints/pretrained_backbones/aptos_lightweight_backbone.pt
output_ckpt: checkpoints/student/dual_csd/best.pt
view_mode: dual
alpha: 0.5
beta: 0.5
lambda_aux: 0.5
csd_variant: smoothl1          # smoothl1 | direction_magnitude | kl_softmax (ablasi)
tau_kd: 2.0
tau_csd: 0.5

# configs/student_dual_logitkd.yaml -- SAMA, KECUALI:
run_name: dual_logitkd
output_ckpt: checkpoints/student/dual_logitkd/best.pt
alpha: 0.5
beta: 0.0
view_mode: dual

# configs/student_dual_no_distill.yaml -- SAMA, KECUALI:
run_name: dual_no_distill
output_ckpt: checkpoints/student/dual_no_distill/best.pt
alpha: 0.0
beta: 0.0
view_mode: dual
# lambda_aux TETAP 0.5 (bukan 0!) -- aux loss tetap jalan di sini (Bagian 5.2)

```

```
# configs/student_macula_only.yaml -- SAMA, KECUALI:
run_name: macula_only
output_ckpt: checkpoints/student/macula_only/best.pt
alpha: 0.0
beta: 0.0
view_mode: macula_only
# lambda_aux TIDAK RELEVAN (single-view tidak punya aux head)

# configs/student_disc_only.yaml -- analog dengan macula_only
```

8.5 Menjalankan Multi-Seed

```
# Kondisi inti (WAJIB 3 seed)
python src/train_student.py configs/student_dual_no_distill.yaml 42,123,2026
python src/train_student.py configs/student_dual_logitkd.yaml 42,123,2026
python src/train_student.py configs/student_dual_csd.yaml 42,123,2026

# Kondisi sekunder (1 seed cukup)
python src/train_student.py configs/student_macula_only.yaml 42
python src/train_student.py configs/student_disc_only.yaml 42
```

8.6 Grid Search alpha/beta/csd_variant

Sama seperti v1, tapi sekarang grid juga mencakup csd_variant (Bagian 5.4.2). Semua kombinasi dijalankan HANYA pada 1 seed dulu (42) untuk memilih kombinasi terbaik di Set B, baru kombinasi terpilih dijalankan penuh 3-seed:

csd_variant	α	β	Catatan
smoothl1	0.5	0.5	default v2
smoothl1	0.5	0.7	CSD lebih dominan
direction_magnitude	0.5	0.5	ablasi arah+besaran terpisah
kl_softmax	0.5	0.5	ablasi v1 (pembanding)

9. Pipeline Evaluasi & Metrik — Revisi v2

9.1 Daftar Metrik (v2 — tambah OVR, latency median/p95)

Metrik	Fungsi/Library	Catatan
QWK (primary)	<code>sklearn.metrics.cohen_kappa_score(weights='quadratic')</code>	Metrik utama
MAE ordinal	<code>mean(abs(y_true-y_pred))</code>	
Severe error rate	<code>mean(abs(y_true-y_pred)>=2)</code>	
Macro-F1	<code>f1_score(average='macro')</code>	
Per-grade sensitivity	<code>recall_score(average=None)</code>	Fokus Grade 1 & 2
Ordinal Violation Rate (OVR)	Bagian 5.6	NEW v2 — wajib ≈ 0 dgn CORALHead monotonic
Dual-view gain (G)	Bagian 5.7	Hanya valid jika aux head disupervisi (Bagian 5.2)
Model size	<code>os.path.getsize</code>	MB; untuk INT8 ukur artefak TorchScript, bukan <code>state_dict</code> (Bagian 10)
Param count	<code>sum(p.numel())</code>	
CPU latency median & p95	Bagian 9.3	NEW v2 — bukan hanya mean

9.2 Script Evaluasi (v2)

```
# src/evaluate.py - v2
import time
import numpy as np
import torch
from sklearn.metrics import cohen_kappa_score, f1_score, recall_score

from losses import get_student_output, ordinal_violation_rate # NEW v2 import

@torch.no_grad()
def get_predictions(model, loader, device, view_mode="dual"):
    model.eval()
    preds, targets, all_p = [], [], []
    for batch in loader:
        macula, disc = batch["macula"].to(device), batch["disc"].to(device)
        out = get_student_output(model, macula, disc, view_mode) # NEW v2 -- konsisten dgn training
        p_key = "p_dual" if view_mode == "dual" else "p"
        grade_pred = (out[p_key] > 0.5).sum(dim=1)
        preds.extend(grade_pred.cpu().tolist())
        targets.extend(batch["label"].tolist())
        all_p.append(out[p_key].cpu())
    return np.array(targets), np.array(preds), torch.cat(all_p, dim=0)

def compute_metrics(y_true, y_pred, p_cumulative=None):
    metrics = {
        "QWK": cohen_kappa_score(y_true, y_pred, weights="quadratic"),
        "MAE": float(np.mean(np.abs(y_true - y_pred))),
        "SevereErrorRate": float(np.mean(np.abs(y_true - y_pred) >= 2)),
        "MacroF1": f1_score(y_true, y_pred, average="macro"),
    }
    sens = recall_score(y_true, y_pred, average=None, labels=[0, 1, 2, 3, 4])
```

```

for g in range(5):
    metrics[f"Sensitivity_Grade{g}"] = sens[g]
if p_cumulative is not None:
    metrics["OrdinalViolationRate"] = ordinal_violation_rate(p_cumulative)
return metrics

def compute_dual_view_gain(model, loader, device):
    y_true, pred_dual, p_dual = get_predictions(model, loader, device, "dual")
    _, pred_macula, p_macula = get_predictions(model, loader, device, "macula_only")
    _, pred_disc, p_disc = get_predictions(model, loader, device, "disc_only")

    qwk_dual = cohen_kappa_score(y_true, pred_dual, weights="quadratic")
    qwk_macula = cohen_kappa_score(y_true, pred_macula, weights="quadratic")
    qwk_disc = cohen_kappa_score(y_true, pred_disc, weights="quadratic")
    G = qwk_dual - max(qwk_macula, qwk_disc)
    return {"QWK_dual": qwk_dual, "QWK_macula": qwk_macula, "QWK_disc": qwk_disc,
    "DualViewGain_G": G}

def model_size_mb(path):
    import os
    return os.path.getsize(path) / (1024 ** 2)

def param_count(model):
    return sum(p.numel() for p in model.parameters())

```

9.3 Protokol Latency (v2 — diperjelas)

✓ **DIPERBAIKI DI v2** — v1 hanya mengukur mean latency tanpa melaporkan jumlah thread CPU, tanpa memisahkan single-view vs dual-view, dan tanpa median/p95 (yang lebih tahan outlier dibanding mean). v2 melaporkan protokol lengkap: thread count eksplisit (`torch.set_num_threads(1)`, dilaporkan di paper sebagai kondisi pengukuran), median & p95 dari 100 run, dan latency single-view/dual-view diukur TERPISAH karena `forward_single()` (Bagian 3-4) sekarang benar-benar hanya memproses satu foto.

```

@torch.no_grad()
def measure_cpu_latency(forward_fn, n_runs=100, warmup=20, n_threads=1):
    torch.set_num_threads(n_threads)
    for _ in range(warmup):
        forward_fn()
    times = []
    for _ in range(n_runs):
        t0 = time.perf_counter()
        forward_fn()
        times.append((time.perf_counter() - t0) * 1000)
    times = np.array(times)
    return {"median_ms": float(np.median(times)), "p95_ms": float(np.percentile(times, 95)),
    "n_threads": n_threads}

# Contoh pemakaian untuk dual-view:
# forward_fn = lambda: model(sample_macula, sample_disc)
# Contoh pemakaian untuk single-view:
# forward_fn = lambda: model.forward_single(sample_macula, which="macula")

```

10. Pipeline Quantization (PTQ) — Revisi v2

10.1 Perbaikan: Fuse Modules & Backend Eksplisit

✓ **DIPERBAIKI DI v2** — v1 langsung memanggil `prepare()/convert()` tanpa fuse conv-bn-act dan tanpa mengatur `torch.backends.quantized.engine` secara eksplisit -- untuk eager-mode static quantization PyTorch, ini SANGAT sering gagal atau menghasilkan model yang tidak benar-benar INT8 di seluruh layer (Critical Issue 11). v2 memanggil `fuse_model()` (Bagian 4.1) sebelum `prepare()`, dan mengatur backend eksplisit di awal.

```
# src/quantize.py - v2
import torch
import torch.nn as nn
from torch.ao.quantization import get_default_qconfig, prepare, convert, QuantStub,
DeQuantStub

torch.backends.quantized.engine = "fbgemm"      # NEW v2 -- x86; "qnnpack" utk ARM/mobile

class QuantizableStudent(nn.Module):
    def __init__(self, student):
        super().__init__()
        self.quant_m = QuantStub()
        self.quant_d = QuantStub()
        self.student = student
        self.dequant = DeQuantStub()

    def forward(self, macula, disc):
        macula = self.quant_m(macula)
        disc = self.quant_d(disc)
        out = self.student(macula, disc)
        return {k: self.dequant(v) for k, v in out.items()}

def ptq_int8(student_fp32, calibration_loader, device="cpu"):
    student_fp32 = student_fp32.to(device).eval()
    student_fp32.backbone.fuse_model()           # NEW v2 -- WAJIB sebelum prepare()

    wrapped = QuantizableStudent(student_fp32)
    wrapped.qconfig = get_default_qconfig("fbgemm")
    prepared = prepare(wrapped, inplace=False)

    with torch.no_grad():
        for i, batch in enumerate(calibration_loader):
            prepared(batch["macula"].to(device), batch["disc"].to(device))
            if i >= 50:
                break

    quantized = convert(prepared, inplace=False)
    return quantized
```

10.2 Fallback: FX Graph Mode (jika Eager Mode Bermasalah)

⚠ **BUTUH VALIDASI LEBIH LANJUT** — Eager-mode static quantization di atas (10.1) SUDAH memasukkan perbaikan dari feedback, tapi PyTorch eager-mode secara umum tetap rentan gagal untuk operasi custom (mis. `torch.cat` di titik fusion, `BatchNorm1d` setelah concatenation, dan `sigmoid` di dalam `CORALHead` yang tidak melalui `QuantStub/DeQuantStub` secara implisit). Ini BELUM diuji end-to-end pada model DR-VERGE yang sebenarnya. Jika `prepare()/convert()` di atas gagal atau menghasilkan model yang errornya besar/tidak

masuk akal, **pindah ke FX Graph Mode Quantization** (`torch.ao.quantization.quantize_fx`), yang jauh lebih robust untuk arsitektur custom karena bekerja di level graph, bukan di level eager module. Alokasikan waktu buffer untuk kemungkinan ini (lihat Hari 9 di timeline, Bagian 15).

```
# Fallback -- FX Graph Mode (skeleton, PERLU divalidasi langsung)
from torch.ao.quantization.quantize_fx import prepare_fx, convert_fx
from torch.ao.quantization import get_default_qconfig_mapping

qconfig_mapping = get_default_qconfig_mapping("fbgemm")
example_inputs = (sample_macula, sample_disc) # tuple, bentuk sama seperti forward()

prepared_fx = prepare_fx(student_fp32, qconfig_mapping, example_inputs)
# ... jalankan kalibrasi seperti biasa ...
quantized_fx = convert_fx(prepared_fx)
```

10.3 Pengukuran Ukuran Model INT8 yang Benar

⚠ **BUTUH VALIDASI LEBIH LANJUT** — `torch.save(quantized.state_dict())` TIDAK SELALU merepresentasikan ukuran deployment INT8 yang sebenarnya secara akurat (`state_dict` bisa menyimpan metadata quantization tambahan). Untuk klaim ukuran model di paper, verifikasi juga dengan `torch.jit.script()` lalu `torch.jit.save()` model quantized (format TorchScript) dan ukur file itu -- ini lebih dekat ke ukuran artefak deployment nyata. Bandingkan dua angka ini; jika berbeda jauh, laporkan keduanya dengan jelas metodologinya di paper.

11. Matriks Eksperimen & Urutan Eksekusi — Revisi v2

11.1 Kondisi & Jumlah Seed

#	Condition ID	Seed	Menjawab
0	smoke_test	1 (42)	NEW v2 -- prasyarat wajib sebelum semua training (Bagian 11.4)
1	pretrain_aptos (resnet50 & lightweight)	1 (42)	NEW v2 -- prasyarat teacher & student
2	teacher	1 (42)	Upper bound
3	macula_only	1 (42)	Single-view baseline (sekarang benar-benar single-view)
4	disc_only	1 (42)	Single-view baseline
5	dual_no_distill	3 (42,123,2026)	Dual-view tanpa transfer
6	dual_logitkd	3 (42,123,2026)	Baseline KD standar
7	dual_csd	3 (42,123,2026)	DR-VERGE — jawaban RQ1
8	dual_csd_int8_ptq	1 (dari seed terbaik #7)	DR-VERGE — jawaban RQ2

11.2 Script Orkestrasi (v2)

```
#!/bin/bash
set -e

echo "=== [0/8] Smoke test (WAJIB lolos dulu) ==="
python src/smoke_test.py

echo "=== [1/8] Pretraining APTOS (resnet50 utk teacher, lightweight utk student) ==="
python src/pretrain_aptos.py configs/pretrain_aptos_resnet50.yaml
python src/pretrain_aptos.py configs/pretrain_aptos_lightweight.yaml

echo "=== [2/8] Training Teacher ==="
python src/train_teacher.py

echo "=== GATE 2 CHECK -- lihat Bagian 14. Jangan lanjut jika gagal. ==="
python src/check_gate2_teacher.py

echo "=== [3/8] macula_only, disc_only (1 seed) ==="
python src/train_student.py configs/student_macula_only.yaml 42
python src/train_student.py configs/student_disc_only.yaml 42

echo "=== [4/8] dual_no_distill (3 seed) ==="
python src/train_student.py configs/student_dual_no_distill.yaml 42,123,2026

echo "=== [5/8] dual_logitkd (3 seed) ==="
python src/train_student.py configs/student_dual_logitkd.yaml 42,123,2026

echo "=== [6/8] dual_csd -- grid search 1 seed dulu, lalu 3 seed pada kombinasi terbaik ==="
python src/train_student.py configs/student_dual_csd.yaml 42
# (pilih csd_variant/alpha/beta terbaik dari Set B, update config, lalu:)
python src/train_student.py configs/student_dual_csd.yaml 123,2026

echo "=== [7/8] PTQ INT8 dari dual_csd seed terbaik ==="
python src/quantize.py

echo "=== [8/8] Evaluasi final SELURUH kondisi ==="
python src/run_final_evaluation.py
```

```
python src/aggregate_seeds.py

echo "SELESAI. Cek results/metrics/all_conditions_raw.csv dan _aggregated.csv"
```

11.3 Perbaikan Script Evaluasi Final (Critical Issue 12)

✓ **DIPERBAIKI DI v2** — v1 memakai `csv.DictWriter(f, fieldnames=rows[0].keys())` -- karena baris single-view TIDAK punya kolom `QWK_dual/QWK_macula/QWK_disc/DualViewGain_G` sementara baris dual-view PUNYA, `DictWriter` akan error begitu baris dengan kolom berbeda ditulis. v1 juga tidak memasukkan teacher dan hasil INT8 ke evaluator akhir walau matriks menyebutkan 7(+) kondisi. v2 mendefinisikan schema kolom eksplisit dengan NaN-fill, dan memasukkan SEMUA kondisi termasuk teacher & INT8.

```
# src/run_final_evaluation.py - v2
import torch, csv, math
from torch.utils.data import DataLoader
from datasets import DeepDRIDDualViewDataset, eval_transform
from models import DualViewLightStudent, DualViewResNetTeacher
from evaluate import get_predictions, compute_metrics, compute_dual_view_gain, model_size_mb,
param_count, measure_cpu_latency
from utils import load_config, get_device

FIELDNAMES = [
    # NEW v2 -- schema EKSPLISIT
    "condition", "seed", "QWK", "MAE", "SevereErrorRate", "MacroF1",
    "Sensitivity_Grade0", "Sensitivity_Grade1", "Sensitivity_Grade2",
    "Sensitivity_Grade3", "Sensitivity_Grade4", "OrdinalViolationRate",
    "QWK_dual", "QWK_macula", "QWK_disc", "DualViewGain_G",
    "ModelSize_MB", "ParamCount", "CPU_Latency_median_ms", "CPU_Latency_p95_ms",
]

def blank_row():
    return {k: math.nan for k in FIELDNAMES}

CONDITIONS = [
    # (name, ckpt_glob_or_path, view_mode, is_teacher)
    ("teacher", "checkpoints/teacher/teacher_final.pt", "dual", True),
    ("macula_only", "checkpoints/student/macula_only/best_seed42.pt", "macula_only",
False),
    ("disc_only", "checkpoints/student/disc_only/best_seed42.pt", "disc_only",
False),
]
MULTISEED_CONDITIONS = ["dual_no_distill", "dual_logitkd", "dual_csd"]
SEEDS = [42, 123, 2026]

def evaluate_one(model, test_loader, device, view_mode, condition, seed, ckpt_path):
    row = blank_row()
    row["condition"], row["seed"] = condition, seed
    y_true, y_pred, p_cum = get_predictions(model, test_loader, device, view_mode)
    row.update(compute_metrics(y_true, y_pred, p_cum))
    if view_mode == "dual":
        row.update(compute_dual_view_gain(model, test_loader, device))
    row["ModelSize_MB"] = model_size_mb(ckpt_path)
    row["ParamCount"] = param_count(model)
    sample = next(iter(test_loader))
    if view_mode == "dual":
        fn = lambda: model(sample["macula"][:1].to(device), sample["disc"][:1].to(device))
    else:
        img = sample["macula"] if "macula" in view_mode else sample["disc"]
        which = "macula" if "macula" in view_mode else "disc"
        fn = lambda: model.forward_single(img[:1].to(device), which=which)
    lat = measure_cpu_latency(fn)
```

```

    row["CPU_Latency_median_ms"] = lat["median_ms"]
    row["CPU_Latency_p95_ms"] = lat["p95_ms"]
    return row

def main():
    device = get_device()
    test_ds = DeepDRiDDualViewDataset("data/splits/deepdrid_setC.csv", "data/raw/deepdrid",
eval_transform)
    test_loader = DataLoader(test_ds, batch_size=16, shuffle=False)

    rows = []

    for name, ckpt_path, view_mode, is_teacher in CONDITIONS:
        Model = DualViewResNetTeacher if is_teacher else DualViewLightStudent
        model = Model(num_classes=5).to(device)
        state = torch.load(ckpt_path, map_location=device)
        model.load_state_dict(state["model_state"] if "model_state" in state else state)
        rows.append(evaluate_one(model, test_loader, device, view_mode, name, "-",
ckpt_path))

    for name in MULTISEED_CONDITIONS:
        for seed in SEEDS:
            ckpt_path = f"checkpoints/student/{name}/best_seed{seed}.pt"
            model = DualViewLightStudent(num_classes=5).to(device)
            state = torch.load(ckpt_path, map_location=device)
            model.load_state_dict(state["model_state"])
            rows.append(evaluate_one(model, test_loader, device, "dual", name, seed,
ckpt_path))

    # INT8 -- dievaluasi terpisah krn model quantized butuh forward khusus (Bagian 10)
    # ... (ditambahkan sesuai kebutuhan, mengikuti pola yang sama) ...

    with open("results/metrics/all_conditions_raw.csv", "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=FIELDNAMES)
        writer.writeheader()
        writer.writerows(rows)

if __name__ == "__main__":
    main()

```

11.4 Smoke Test (BARU v2 — Wajib Sebelum Full Training)

✓ **DIPERBAIKI DI v2** — Ditambahkan sesuai rekomendasi review: satu batch data dijalankan lewat SELURUH komponen (model teacher, model student, semua loss, backward pass) sebelum commit ke training penuh berjam-jam. Ini menangkap sebagian besar bug shape/tipe data dalam hitungan detik, bukan setelah training 2 jam gagal di epoch terakhir.

```

# src/smoke_test.py – NEW v2
import torch
from torch.utils.data import DataLoader
from datasets import DeepDRiDDualViewDataset, eval_transform
from models import DualViewResNetTeacher, DualViewLightStudent
from losses import combined_student_loss, get_student_output, ordinal_violation_rate
from utils import get_device

def smoke_test():
    device = get_device()
    ds = DeepDRiDDualViewDataset("data/splits/deepdrid_train.csv", "data/raw/deepdrid",
eval_transform)
    loader = DataLoader(ds, batch_size=4, shuffle=True)

```

```

    batch = next(iter(loader))
    macula, disc, y = batch["macula"].to(device), batch["disc"].to(device),
batch["label"].to(device)

    teacher = DualViewResNetTeacher().to(device)
    student = DualViewLightStudent().to(device)

    teacher_out = teacher(macula, disc)
    assert teacher_out["p_dual"].shape == (4, 4), teacher_out["p_dual"].shape
    assert ordinal_violation_rate(teacher_out["p_dual"]) == 0.0, "CORAL monotonicity gagal!"

    for view_mode in ["dual", "macula_only", "disc_only"]:
        student_out = get_student_output(student, macula, disc, view_mode)
        loss, log = combined_student_loss(teacher_out, student_out, y, view_mode, alpha=0.5,
beta=0.5)
        loss.backward()
        print(f"[{view_mode}] OK -- loss={loss.item():.4f}, log={log}")
        student.zero_grad()

    print("SMOKE TEST PASSED.")

if __name__ == "__main__":
    smoke_test()

```

12. Reproducibility, Logging & Config Management

- Seed dikunci per-run (Bagian 8.2), dan sekarang checkpoint diberi suffix `_seed<n>.pt` supaya 3 seed pada kondisi inti tidak saling menimpa.
- **Kejujuran split** (Bagian 2.1) dicatat di README repo: apakah split yang dipakai resmi atau custom, dengan tanggal & cara verifikasinya.
- Checkpoint pretraining APTOS (`checkpoints/pretrained_backbones/`) di-commit metadata-nya (val QWK, epoch, tanggal) supaya jelas checkpoint mana yang dipakai tiap eksperimen.
- `aggregate_seeds.py` (Bagian 11) menghasilkan tabel `mean±std` per kondisi — inilah yang dilaporkan di paper untuk `dual_no_distill`, `dual_logitkd`, `dual_csd`, bukan angka seed tunggal.

```
# src/aggregate_seeds.py – NEW v2
import pandas as pd

def aggregate():
    df = pd.read_csv("results/metrics/all_conditions_raw.csv")
    numeric_cols = ["QWK", "MAE", "SevereErrorRate", "MacroF1", "DualViewGain_G"]
    agg = df.groupby("condition")[numeric_cols].agg(["mean", "std"])
    agg.to_csv("results/metrics/all_conditions_aggregated.csv")
    print(agg)

if __name__ == "__main__":
    aggregate()
```

13. Checklist Titik Rawan Bug — Update v2

Checklist v1 dipertahankan dan diperbarui. Baris bertanda ✓ adalah bug v1 yang sudah diperbaiki desainnya di v2 (tetap perlu dicek ulang saat implementasi nyata); baris baru ditambahkan berdasarkan feedback.

Gejala	Kemungkinan Penyebab	Perbaikan / Status
QWK macula_only/disc_only jelek meski model 'terlihat' training normal	v1: view_mode tidak benar-benar dipakai (Critical Issue 1)	✓ v2: get_student_output() + forward_single() memaksa single-view sungguhan
Dual-view gain (G) besar mencurigakan pada dual_no_distill/dual_logitkd	v1: aux head tidak disupervisi (Critical Issue 2)	✓ v2: L_aux wajib di semua kondisi dual-view
OrdinalViolationRate > 0	CORALHead lama (bias bebas) ATAU bug pada _ordered_biases()	✓ v2: ordered-bias parameterization; jika masih >0, cek implementasi cumsum/softplus
L_CSD turun cepat ke ~0 tapi QWK tidak membaik	Kemungkinan CSD variant 'smoothl1' terlalu mudah dipuaskan (delta kecil ditiru sempurna tapi tidak informatif) -- coba direction_magnitude	Bandingkan 3 varian (Bagian 5.4.2, 8.6) di Set B
Val loss NaN saat fine-tuning backbone	LR fine-tuning terlalu tinggi	Turunkan ke 1e-6 atau tambah gradient clipping
Data leakage (val QWK tidak masuk akal tinggi)	Split per-gambar bukan per-pasien, ATAU tercampur official/custom (Bagian 2.1)	Verifikasi set(train.patient_id) & set(test.patient_id) == set()
Static PTQ error / model INT8 tidak lebih kecil	fuse_model() tidak dipanggil, backend belum diatur (Critical Issue 11)	✓ v2: Bagian 10.1; jika tetap gagal, pindah ke FX Graph Mode (10.2)
run_final_evaluation.py crash 'dict contains fields not in fieldnames'	DictWriter tanpa schema eksplisit (Critical Issue 12)	✓ v2: FIELDNAMES eksplisit + NaN-fill (Bagian 11.3)
Teacher checkpoint ternyata bukan yang terbaik	best_val_loss dihitung tapi tidak dipakai simpan (Critical Issue 9)	✓ v2: simpan berdasarkan val QWK + early stopping (Bagian 6.1)
Backbone teacher/student memakai checkpoint APTOS yang salah / tertukar	Kedua arsitektur BERBEDA, tidak bisa share checkpoint (Critical Issue 10)	✓ v2: dua config pretrain terpisah (Bagian 7.1), cek backbone_type di config
QWK sangat tinggi di Set B tapi jelek di Set C	Overfitting hyperparameter ke Set B lewat grid search berulang	Batasi jumlah kombinasi grid (Bagian 8.6); jangan intip Set C sebelum evaluasi final

14. Go/No-Go Gates (BARU v2)

Diusulkan dalam review teknis dan diadopsi penuh — lima gate ini adalah pemeriksaan wajib sebelum melanjutkan ke tahap berikutnya. **Jangan lanjut ke tahap berikutnya jika sebuah gate gagal** — perbaiki dulu tahap sebelumnya, karena melanjutkan di atas fondasi yang cacat akan mencemari semua hasil di atasnya.

Gate 1 — Dataset

- Pairing macula-disc per mata benar (dicek manual pada sampel acak).
- Tidak ada patient overlap antar-split (Bagian 2.1, 12).
- Distribusi grade per split dicatat dan dilaporkan (untuk konteks imbalance, Bagian 2.4).
- Semua path file di CSV split valid (tidak ada file hilang).

Gate 2 — Teacher

Sebelum mulai training student sama sekali:

```
QWK_dual(teacher) > max(QWK_macula(teacher), QWK_disc(teacher))
```

dan kedua auxiliary head teacher (p_macula, p_disc) harus punya QWK yang masuk akal (jauh di atas performa acak/mayoritas-kelas). **Jika dual-view gain teacher sendiri tidak positif, JANGAN lanjut ke training CSD** — perbaiki dulu teacher (kemungkinan: lambda_aux terlalu kecil, epoch kurang, atau data leakage yang membuat single-view sudah 'terlalu mudah').

Gate 3 — Sinyal CSD

Dihitung pada satu batch validasi sebelum training CSD penuh dimulai:

- Mean $|\delta^T|$ (L1 norm rata-rata) — kalau mendekati 0 di seluruh sampel, berarti teacher tidak menunjukkan sinyal complementarity yang bisa didistilasi (kembali ke Gate 2).
- Distribusi δ^T per-threshold — cek apakah shift terkonsentrasi di threshold tertentu (masuk akal) atau tersebar acak (mencurigakan).
- Persentase sampel dengan shift non-trivial (mis. $|\delta^T| > 0.02$) — kalau sangat rendah, CSD kemungkinan tidak akan banyak membantu dibanding logit KD biasa.

Gate 4 — Eksperimen Utama (RQ1)

DR-VERGE (dual_csd) idealnya, berdasarkan mean 3-seed di Set B, harus:

- Mengungguli dual_no_distill.
- Kompetitif dengan atau lebih baik dari dual_logitkd.
- Mempertahankan dual-view gain (G) lebih dekat ke G milik teacher, dibanding baseline lain.
- Tidak menaikkan severe error rate dibanding baseline.

⚠ BUTUH VALIDASI LEBIH LANJUT — Jika CSD TIDAK memenuhi kriteria ini setelah grid search (Bagian 8.6) dan 3-seed, itu sendiri adalah TEMUAN VALID yang layak dilaporkan jujur di paper (dengan analisis kenapa) — bukan

berarti eksperimen gagal secara keseluruhan. RQ1 tetap terjawab (jawabannya bisa 'tidak', dengan penjelasan), yang penting protokolnya kredibel.

Gate 5 — PTQ

- Pastikan model benar-benar ter-convert ke INT8 di layer Conv2d (cek lewat `print(quantized_model)` — layer harus muncul sebagai `QuantizedConv2d`, bukan `Conv2d` biasa).
- Bukan sekadar wrapper FP32 dengan `QuantStub/DeQuantStub` yang tidak benar-benar mengkonversi bobot.
- Ukuran file diverifikasi lewat `TorchScript` (Bagian 10.3), bukan hanya `state_dict`.

15. Timeline 10 Hari & Pembagian Kerja — Revisi v2 (Tim 3 Orang)

Scope bertambah signifikan di v2 (aux loss, imbalance handling, monotonic CORAL, pretraining nyata, multi-seed, gates). Dengan tim sekarang 3 orang, sebagian jalur bisa diparalelkan — tapi Gate 2 (Bagian 14) tetap jadi titik sinkronisasi wajib sebelum training student dimulai.

Hari	Target	Paralel?	Output
1	Setup repo, environment, patient-wise split (cek dulu apakah ada split resmi!), preprocessing	Bersama	Split tervalidasi Gate 1
2	Pretraining APTOS: resnet50 (utk teacher) + lightweight (utk student)	Paralel (2 orang, 1 model masing-masing)	2 checkpoint backbone
3–4	Training teacher (freeze → fine-tune, dgn aux loss + pos_weight)	1 orang fokus; 2 orang lain siapkan smoke test & script evaluasi	teacher_final.pt lolos Gate 2
5	Smoke test lolos (Bagian 11.4); baseline macula_only, disc_only	Paralel: 1 orang smoke test, 2 orang baseline	2 checkpoint + smoke test hijau
6	dual_no_distill (3 seed) + dual_logitkd (3 seed) — bisa dijalankan paralel di 2 mesin/GPU berbeda	Paralel (2 orang, tiap orang pegang 1 kondisi × 3 seed)	6 checkpoint (2 kondisi × 3 seed)
7	Implementasi CSD (3 varian loss, Bagian 5.4.2) + Gate 3 check + smoke test ulang	1 orang implementasi, 1 orang validasi Gate 3, 1 orang siapkan eval script	csd_loss() lolos Gate 3
8	Grid search csd_variant/alpha/beta (1 seed) → training dual_csd penuh (3 seed)	Bersama (grid search bisa dibagi kombinasi antar-anggota)	dual_csd terbaik, 3 seed
8.5	PTQ INT8 (fuse_model + eager mode; fallback FX-mode jika perlu, Bagian 10)	1 orang fokus penuh (risiko debugging tertinggi)	Lolos Gate 5 atau dilaporkan sbg future work
9	Evaluasi final SEMUA kondisi ke Set C (SEKALI), aggregate_seeds.py, cek Gate 4	Bersama	all_conditions_aggregated.csv lengkap
10	Penulisan artikel, tabel & grafik	Dibagi per bagian paper	Draft KTI lengkap

15.1 Aturan Emas (tidak berubah, ditegaskan lagi)

- **Set C hanya disentuh SEKALI**, di Hari 9, untuk semua kondisi × semua seed sekaligus.
- **Setiap gate (Bagian 14) adalah checkpoint wajib** — jangan lanjut ke tahap berikut sebelum gate sebelumnya hijau, meskipun tim tergoda mengejar waktu.
- Kalau PTQ (Hari 8.5) tidak lolos Gate 5 setelah mencoba eager-mode dan FX-mode, laporkan sebagai future work — RQ1 (Gate 4) tetap harus tuntas karena itu kontribusi inti.
- Dengan 3 orang, godaan terbesar adalah paralelisasi berlebihan yang membuat split/checkpoint/config saling tidak sinkron — **selalu commit config & split ke repo bersama sebelum menjalankan eksperimen paralel apa pun**, jangan biarkan tiap anggota punya versi lokal yang berbeda.

Lampiran A — Ringkasan Notasi (v2)

Simbol	Arti
z_m, z_d	Vektor fitur backbone untuk macula, disc
z_{fused}	Konkatenasi z_m, z_d setelah BatchNorm
$p_{dual}, p_{macula}, p_{disc}$	$P(y>k), k=0..3$ — cumulative threshold probabilities, BUKAN distribusi 5-kelas
p_{agg}	Rata-rata p_{macula} dan p_{disc}
Δ	$p_{dual} - p_{agg}$ — cumulative ordinal decision boundary shift
pos_weight_k	NEW v2 — bobot class-imbalance per-threshold
λ_{aux}	NEW v2 — bobot auxiliary loss (macula+disc head) pada model dual-view
$csd_variant$	NEW v2 — smoothl1 (default) direction_magnitude kl_softmax (ablasi)
OVR	NEW v2 — Ordinal Violation Rate, sanity-check monotonicity
α, β	Bobot L_{logit_KD} dan L_{CSD}
G	Dual-view gain: $QWK_{dual} - \max(QWK_{macula}, QWK_{disc})$

Lampiran B — Sumber Dataset & Kejujuran Metodologi

- **DeepDRiD**: sumber utama training & evaluasi. Split WAJIB diverifikasi resmi vs custom (Bagian 2.1) sebelum eksperimen pertama.
- **APTOS 2019**: pretraining backbone saja (dua backbone terpisah, Bagian 7).
- **DRTiD (CrossFit, 2022)**: HANYA rujukan angka literatur, tidak dipakai sebagai data tambahan.

Lampiran C — Daftar Konsolidasi "Butuh Validasi Lebih Lanjut"

Kumpulan seluruh flag kuning di dokumen ini, dikumpulkan di satu tempat sebagai checklist validasi awal proyek:

1. Apakah metadata DeepDRiD punya kolom partisi resmi (Bagian 2.1)? Jika ya, split custom harus diganti.
2. Nama kolom CSV asli (patient_id, macula_path, dst.) belum dicek terhadap file metadata asli (Bagian 2.2).
3. Apakah horizontal flip aman secara klinis untuk skema DR grading ini, atau berisiko mengaburkan makna laterality (Bagian 2.3)?
4. Apakah distribusi grade di DeepDRiD cukup timpang sehingga WeightedRandomSampler perlu ditambahkan di atas pos_weight (Bagian 2.4)?
5. Konversi CSD ke distribusi 5-kelas eksplisit (q0..q4) adalah opsi masa depan yang belum diimplementasikan/divalidasi (Bagian 5.4).
6. Kapasitas backbone student (~0,3–0,4M parameter) berisiko jadi bottleneck — pantau di Hari 7-8, siapkan fallback MobileNetV3/EfficientNet-B0 (Bagian 4.3).
7. Static PTQ eager-mode (Bagian 10.1) belum diuji end-to-end pada arsitektur DR-VERGE sebenarnya — siapkan waktu buffer untuk pindah ke FX Graph Mode (Bagian 10.2) jika perlu.
8. Ukuran model INT8 harus diverifikasi lewat TorchScript, bukan hanya state_dict (Bagian 10.3).

— *Akhir Dokumen (v2)* —